SAFE DRIVER ASSISTANT

AUTHORS:

NAME	SRN	BRANCH	SEMESTER
SHASHANK S BHARADWAJ	PES1UG19EC278	*ELECTRONICS AND COMMUNICATION*	IV
PRUTHVI G KARIGIRI	PES1UG19CS354	*COMPUTER SCIENCE*	IV
KARTHIK RAJENDRA PRASAD	PES1UG19CS209	*COMPUTER SCIENCE*	IV
BHAVANA SHIVAKUMAR	PES1UG19EE024	*ELECTRICAL AND ELECTRONICS*	IV

MENTOR: **Dr. Chandar T S**

TEAM ID: *T-15*

SESSION: JANUARY- MAY 2021

Contents:

1. Abstract of the project

2. Introduction

2.1 Motivation

2.2 Problem statement

2.3 Survey

3. Components required

4. Methodology

5. Work carried out

6. Results and discussion

7. Conclusion and future improvement capabilities

8. References

Appendices

1. Appendix A

2. Appendix B

1. ABSTRACT

This project is a combination of both electronics and computer science engineering concepts. This module uses Raspberry Pi microcontroller, infrared cameras *(night vision)* and facial recognition algorithms to help us detect drowsiness in the face of the driver of the vehicle.

This system has also been integrated with an MQ-3 alcohol sensor module to help identify if the driver is intoxicated. This sensor can be pinned to the seat belt of the car so it can inspect the air exhaled by the driver.

The safe driver assistant is only a humble attempt towards creating a more civilized demographic. The duties of the person behind the wheel are extremely essential. The project is one of the ways to ensure safe travel.

2. INTRODUCTION

2.1 Motivation

The number of accidents while driving is on a constant rise but it can be significantly reduced with the help of modern technology. By constant monitoring of the physical condition of the driver, the appropriate precautionary measures can be taken to ensure the safety of the driver. The two most common conditions that impair the driver are drowsiness and intoxication.

Detection of drowsiness can be done by constantly monitoring the driver's face for things like eye movement (faster blinking of eyes is a sign of drowsiness), yawning and then can provide an audible alarm to alert the driver or even force the driver to pull over the vehicle for rest.

Intoxication can be monitored with the help of a breathalyzer. This sensor can be mounted somewhere in the cabin (For example, the seat belt) and then allow the vehicle to start only if the blood alcohol levels of the driver is within the safe limit.

2.2 Problem statement

Drowsy driver detection using image processing with Raspberry Pi integrated with a camera module (OV5647) with an alcohol sensor and aheart rate sensor for additional driver impairity prediction.

2.3 Survey

According to CDC, an estimated 1 in 20 adults drivers report having fallen asleep while driving every month. The NHTSA estimates that drowsy driving was responsible for 72,000 crashes, 44,000 injuries and 800 deaths in 2013 alone. However, these numbers are underestimated and up to 6,000 fatal crashes each year may be caused due to drowsy drivers.

In another survey conducted, A majority of drivers (58.6%) admitted that they occasionally drive while fatigued or drowsy. Of greater importance, 14.5% of respondents admitted that they had fallen asleep or “nodded off” while driving during the past year. Nearly 2% were involved in a fatigue or drowsy driving related crash in the past year.

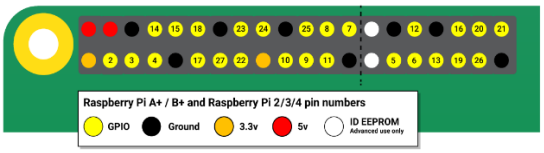
Drinking and driving is one of the main causes of road crashes worldwide. In high-income countries about 20% of fatally injured drivers have excess alcohol in their blood, while in some low- and middle-income countries these figures may be up to 69%. It is estimated that 5–35% of all road deaths are reported as alcohol-related. Driving after drinking alcohol significantly increases the risk of a crash and the severity of that crash

From the evidence collected from the literature survey, EAR – eye aspect ratio is easy and a fairly efficient way of determining the status of the driver.

3. COMPONENTS REQUIRED

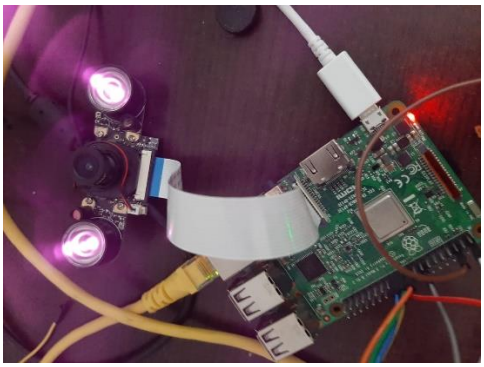
3.1. Raspberry Pi 3 –

- *1.4GHz 64-bit quad-core processor*
- *Dual-band wireless LAN*
- *Bluetooth 4.2/BLE*
- *Faster Ethernet*
- *CSI camera port for connecting a Raspberry Pi camera*
- *5V/2.5A DC power input*



3.2. Pi Camera Module (OV5647)

- *Embedded IR-CUT filter eliminating colour distortion in the daylight Comes with infrared LED*
- *supports night vision*
- *5-megapixel OV5647 sensor*

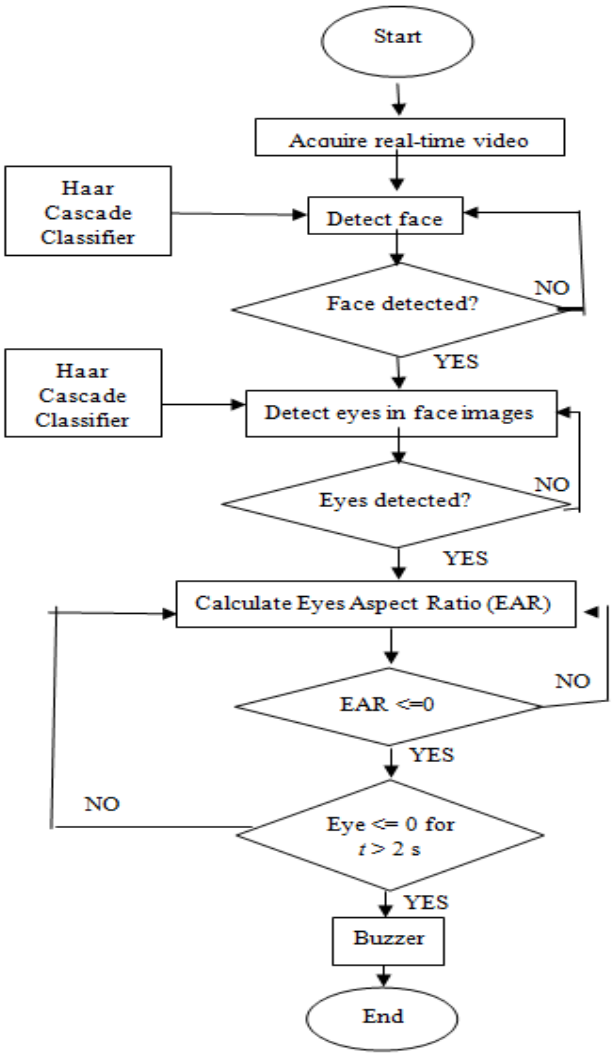


3.3. Micro USB Cable – *Power input*

3.4. Buzzer

3.5. Sensor to detect alcohol: MQ3 sensor (detects the trace of ethanol in air).

3.6. Heart rate sensor – *An optical heart rate sensor measures pulse waves, which are changes in the volume of a blood vessel that occur when the heart pumps blood.*

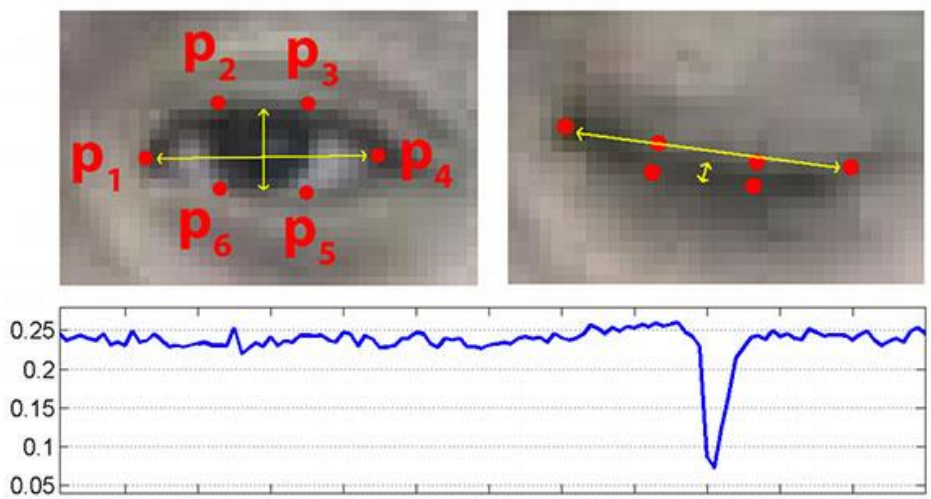


4. METHODOLOGY

I. Capture he subject's face with the camera module.
II. Place points on the image.
III. Check for eyes closing and/or yawning.
IV. Check for blood alcohol levels.
V. Check for heart rate to determine drowsiness and any irregularities.

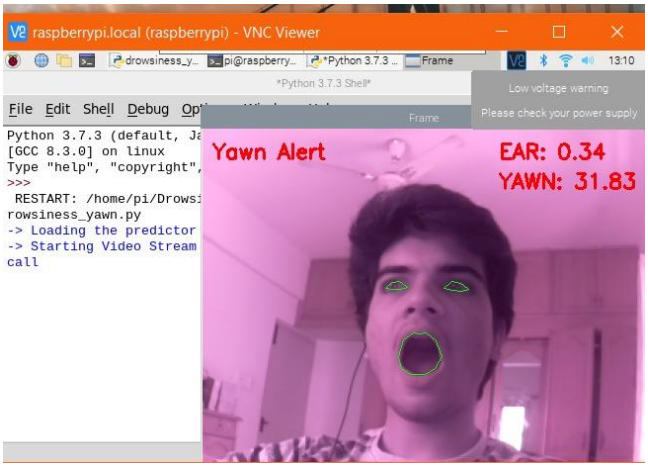
Driver drowsiness detection is designed mainly to keep the driver awake while driving to avoid the accident due to sleepiness. The alert signal is generated from embedded device to awake driver from sleepy state.

The Pi along with Raspbian camera is used to calculate the drowsiness of the driver in real time. Fatigue is measured by detecting Eye and face using Haar Cascade Classifier, especially facial landmarks is detected using shape-predictor and Eye Aspect Ratio (EAR) by calculating the Euclidean distance between the eyes.

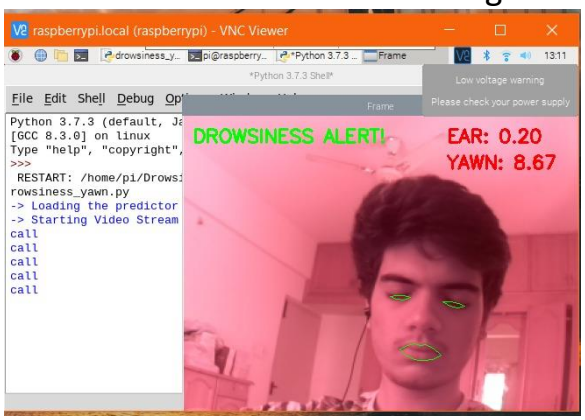


Accurate eye detection and faces in every frame will help to calculate drowsiness level. Frequent detection of eye blinking and head tilting is

measured properly and it helps to indicates drowsiness. When he/she reaches maximum threshold, the driver will be alarmed by a loud warning that will wake up the driver from the sleep state.



This alcohol sensor efficiently checks drunken driving. By implementing this design, a safe car journey is possible decreasing the accident rate due to drinking. By implementing this design, drunken drivers can be controlled so are the accidents due to drunken driving.



5. WORK CARRIED OUT

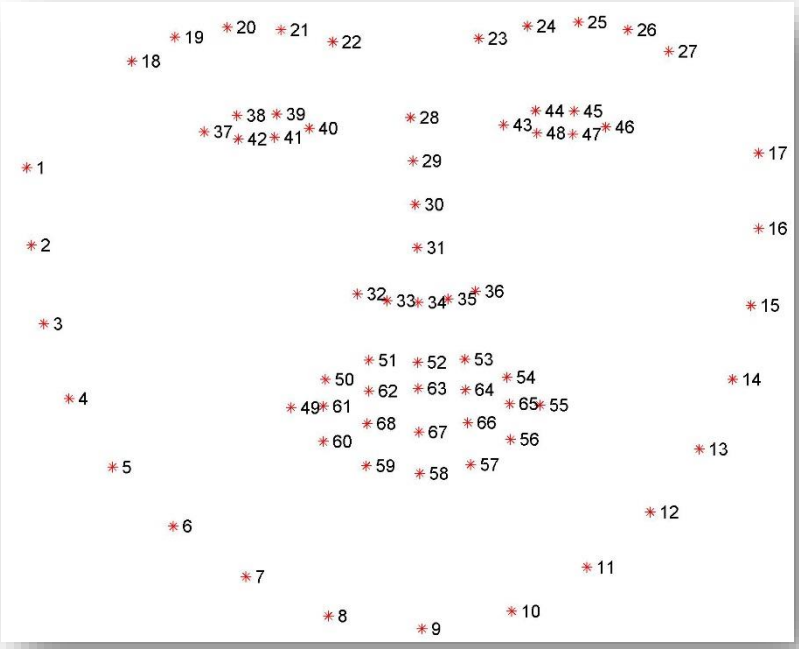
I. Setup of Raspberry pi. – Installing Raspbian operating system.

II. Testing of Raspbian OS with the required libraries
- *OpenCV*
- *Dlib*
- *RPi.GPIO*
- *Adafruit_MCP3008*
- *NumPy*

III. Testing of the breath alcohol sensor (MQ3) and the heart rate sensor.

IV. Coding and integration of the heart rate sensor.

V. Coding and testing of the night vision camera for efficient performance during both day and night. Calibrating the **EAR threshold values** and the Euclidian distance between the eye and mouth landmarks.



To start, we will apply **OpenCV's Haar cascades** to detect the face in an image, which boils down to finding the bounding box (x, y)-coordinates of the face in the frame.

Given the bounding box the face we can apply **dlib's facial landmark predictor** to obtain 68 salient points used to localize the eyes, eyebrows, nose, mouth, and jawline

6. RESULTS AND DISCUSSIONS

This study attempted to address the issue by calculating the level of drowsiness and depending on whether there is presence of alcohol, send an appropriate alert.. A requirement for this paper was the utilisation of a **Raspberry Pi Camera and Raspberry Pi 3 module**, which were able to calculate the level of drowsiness in drivers. The frequency of head tilting and blinking of the eyes was used to determine whether or not a driver felt drowsy.

The driver drowsiness can be measured using EAR. The ratio of the eye can vary for each and every person. One is calculated for eye-opening condition and another one for eye closing condition. Eye closing rate is measured after every 0.5 seconds and if the value crosses already existed threshold value, then the raspberry pi 3 receives the alert signal from alarm connected to the GPIO pins of Pi 3 board.

7. CONCLUSIONS AND FUTURE WORK

The drowsiness detection and correction system developed here is capable of detecting drowsiness in a rapid manner. When the person closing his eyes for more than fixed threshold range then the alert signal is generated to wake up the driver from sleepy state and also through the cloud service the alert message can be sent to the owner of the car along with the car plate number.

The system can be commercially generalized and well employed in today's vehicles with comparatively fewer expenses. As a whole, the system when equipped with the vehicles can reduce the traffic collisions occurring, related dangers and expenses in our country. Government must enforce laws to install such circuit in every car and must regulate all car companies to preinstall such mechanisms while manufacturing the car itself. If this is achieved the deaths due to drunken drivers can be brought to minimum level. In this type of system, future scope can be safely landing of car aside without disturbing other vehicles. The alcohol breath detector can be setup in a way that the car refuses to start if the driver is intoxicated.

8. REFERENCES

1. W. Tipprasert, T. Charoenpong, C. Chianrabutra and C. Sukjamsri, "A Method of Driver's Eyes Closure and Yawning Detection for Drowsiness Analysis by Infrared Camera," 2019 First International Symposium on Instrumentation, Control, Artificial Intelligence, and Robotics (ICA-SYMP), Bangkok, Thailand, 2019, pp. 61-64, doi: 10.1109/ICA-SYMP.2019.8646001.

2. C. V. Anilkumar, M. Ahmed, R. Sahana, R. Thejashwini and P. S. Anisha, "Design of drowsiness, heart beat detection system and alertness indicator for driver safety," 2016 IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology (RTEICT), Bangalore, 2016, pp. 937-941, doi: 10.1109/RTEICT.2016.7807966.

3. A. A. Suhaiman, Z. May and N. A. A.Rahman, "Development of an intelligent drowsiness detection system for drivers using image processing technique," 2020 IEEE Student Conference on Research and Development (SCOReD), Batu Pahat, Johor, Malaysia, 2020, pp. 233-236, doi: 10.1109/SCOReD50371.2020.9250948.

4.L. Jia, D. Zhao, K. Zheng, Z. Li, G. Sun, and F. Zhang, "Smartphone-based fatigue detection system using the progressive locating method", IET Intell. Transp. Syst, vol. 10, no. 3, pp. 148– 156, 2016.

5. J. S. Jayasenan and P. S. Smitha, "Driver Drowsiness Detection System", vol. 4, no. 1, pp. 34–37, 2014.

6.V. Triyanti and H. Iridiastadi, "Challenges in detecting drowsiness based on driver's behavior", IOP Conf. Ser. Mater. Sci. Eng, vol. 277, 2017.

APPENDIX

Appendix A: Code for alcohol sensor

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
GPIO.setup(25,GPIO.IN)
GPIO.setup(23,GPIO.OUT)
GPIO.setwarnings(False)
def buzzerRoutine():
        while True:
                GPIO.output(23,GPIO.HIGH)
                time.sleep(0.5)
                GPIO.output(23,GPIO.LOW)
                time.sleep(0.5)


while True:
        input1 = GPIO.input(25)
        if input1 == 0 :
                print("No alcohol")
                GPIO.output(23,GPIO.LOW)


        else:
                print("alcohol  is detected")
                for i in range(0,20):
                        buzzerRoutine()
                        i+=1
```

Appendix B: Code for image processing

```
from scipy.spatial import distance as dist
from imutils.video import VideoStream
from imutils import face_utils
from threading import Thread
import numpy as np
import argparse
import imutils
import time
```

```
import dlib
import cv2
import os

def alarm(msg):
    global alarm_status
    global alarm_status2
    global saying

    while alarm_status:
        print('call')
        s = 'espeak "'+msg+'"'
        os.system(s)

    if alarm_status2:
        print('call')
        saying = True
        s = 'espeak "' + msg + '"'
        os.system(s)
        saying = False

def eye_aspect_ratio(eye):
    A = dist.euclidean(eye[1], eye[5])
    B = dist.euclidean(eye[2], eye[4])

    C = dist.euclidean(eye[0], eye[3])

    ear = (A + B) / (2.0 * C)

    return ear

def final_ear(shape):
    (lStart, lEnd) = face_utils.FACIAL_LANDMARKS_IDXS["left_eye"]
    (rStart, rEnd) = face_utils.FACIAL_LANDMARKS_IDXS["right_eye"]

    leftEye = shape[lStart:lEnd]
    rightEye = shape[rStart:rEnd]

    leftEAR = eye_aspect_ratio(leftEye)
    rightEAR = eye_aspect_ratio(rightEye)
```

```
    ear = (leftEAR + rightEAR) / 2.0
    return (ear, leftEye, rightEye)

def lip_distance(shape):
    top_lip = shape[50:53]
    top_lip = np.concatenate((top_lip, shape[61:64]))

    low_lip = shape[56:59]
    low_lip = np.concatenate((low_lip, shape[65:68]))

    top_mean = np.mean(top_lip, axis=0)
    low_mean = np.mean(low_lip, axis=0)

    distance = abs(top_mean[1] - low_mean[1])
    return distance


ap = argparse.ArgumentParser()
ap.add_argument("-w", "--webcam", type=int, default=0,
                help="index of webcam on system")
args = vars(ap.parse_args())

EYE_AR_THRESH = 0.3
EYE_AR_CONSEC_FRAMES = 20
YAWN_THRESH = 30
alarm_status = False
alarm_status2 = False
saying = False
COUNTER = 0

print("-> Loading the predictor and detector...")
#detector = dlib.get_frontal_face_detector()
detector = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")
#Faster but less accurate
predictor = dlib.shape_predictor('shape_predictor_68_face_landmarks.dat')


print("-> Starting Video Stream")
```

```
vs= VideoStream(usePiCamera=True).start()
time.sleep(1.0)

while True:

    frame = vs.read()
    frame = imutils.resize(frame, width=450)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    #rects = detector(gray, 0)
    rects = detector.detectMultiScale(gray, scaleFactor=1.1,
            minNeighbors=5, minSize=(30, 30),
            flags=cv2.CASCADE_SCALE_IMAGE)

    #for rect in rects:
    for (x, y, w, h) in rects:
        rect = dlib.rectangle(int(x), int(y), int(x + w),int(y + h))

        shape = predictor(gray, rect)
        shape = face_utils.shape_to_np(shape)

        eye = final_ear(shape)
        ear = eye[0]
        leftEye = eye [1]
        rightEye = eye[2]

        distance = lip_distance(shape)

        leftEyeHull = cv2.convexHull(leftEye)
        rightEyeHull = cv2.convexHull(rightEye)
        cv2.drawContours(frame, [leftEyeHull], -1, (0, 255, 0), 1)
        cv2.drawContours(frame, [rightEyeHull], -1, (0, 255, 0), 1)

        lip = shape[48:60]
        cv2.drawContours(frame, [lip], -1, (0, 255, 0), 1)

        if ear < EYE_AR_THRESH:
            COUNTER += 1

            if COUNTER >= EYE_AR_CONSEC_FRAMES:
```

```
                if alarm_status == False:
                    alarm_status = True
                    t = Thread(target=alarm, args=('wake up ',))
                    t.deamon = True
                    t.start()

                cv2.putText(frame, "DROWSINESS ALERT!", (10, 30),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

        else:
            COUNTER = 0
            alarm_status = False

        if (distance > YAWN_THRESH):
                cv2.putText(frame, "Yawn Alert", (10, 30),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
                if alarm_status2 == False and saying == False:
                    alarm_status2 = True
                    t = Thread(target=alarm, args=('you are sleepy',))
                    t.deamon = True
                    t.start()
        else:
            alarm_status2 = False

        cv2.putText(frame, "EAR: {:.2f}".format(ear), (300, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
        cv2.putText(frame, "YAWN: {:.2f}".format(distance), (300, 60),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)


    cv2.imshow("Frame", frame)
    key = cv2.waitKey(1) & 0xFF

    if key == ord("q"):
        break

cv2.destroyAllWindows()
vs.stop()
```